Submission Worksheet

Submission Data

Course: IT114-450-M2025
Assignment: IT114 - Milestone 3 - RPS
Student: Mariano R. (mr822)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 7/30/2025 12:45:53 AM
Updated: 8/6/2025 9:03:12 PM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-rps/grading/mr822
View Link: https://learn.ethereallab.app/assignment/v3/IT114-450-M2025/it114-milestone-3-rps/view/mr822

Instructions

1. Refer to Milestone3 of Rock Paper Scissors
    1. Complete the features
2. Ensure all code snippets include your ucid, date, and a brief description of what the code does
3. Switch to the `Milestone3` branch
    1. `git checkout Milestone3`
    2. `git pull origin Milestone3`
4. Fill out the below worksheet as you test/demo with 3+ clients in the same session
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone3`
    4. On Github merge the pull request from `Milestone3` to `main`
7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

Section #1: (1 pt.) Core Ui

Progress: 100%

Task #1 (0.50 pts.) - Connection/Details Panels

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the connection panel with valid data
- Show the user details panel with valid data

Connection

User Details

Saved: 8/1/2025 1:25:21 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow from recording/capturing these details and passing them through the connection process

Your Response:

The client captures the user's name through the /name command, which sets it in the myUser object. When the user connects using /connect, the client sends this name to the server. The server responds with a CLIENT_ID payload, confirming the assigned ID and name, which are stored in myUser and added to knownClients. The LoginRoomUI then displays this data by accessing getClientName() and getClientId() from the client, and renders the user in the lobby list and connection label.

Saved: 8/1/2025 1:25:21 AM

Task #2 (0.50 pts.) - Ready Panel

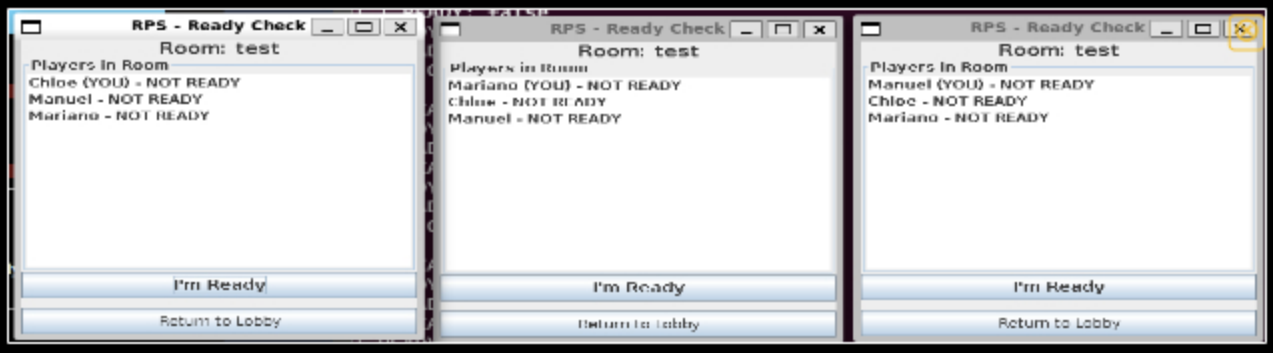
Progress: 100%

Part 1:

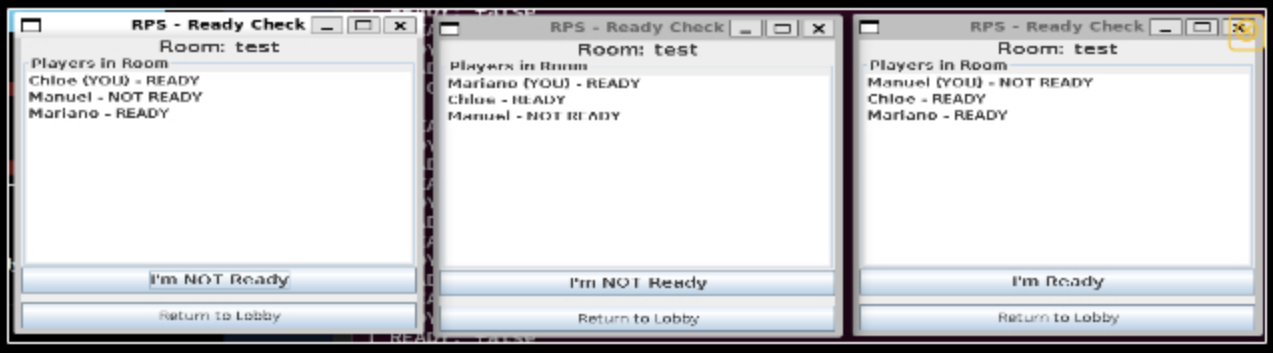
Progress: 100%

Details:

- Show the button used to mark ready
- Show a few variations of indicators of clients being ready (3+ clients)



Ready1



Ready2

Saved: 8/1/2025 3:23:06 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for marking READY from the UI
- Briefly explain the code flow from receiving READY data and updating the UI

Your Response:

When a user clicks "I'm Ready," the handleReadyToggle() method in ReadyCheckUI toggles the isReady flag, sends a PayloadType.READY to the server, and immediately updates their own status in the UI via updateStatusForUser().

When the server receives a READY, it broadcasts a PayloadType.READY_STATUS to all clients. The receiving client processes this in Client.processPayload(), then calls

ReadyCheckUI.updateStatusForUser(), which updates the readyStatusMap and refreshes the UI with the new status in updateUserList().

Saved: 8/1/2025 3:23:06 AM

Section #2: (2 pts.) Project Ui

Progress: 100%

Task #1 (0.67 pts.) - User List Panel

Progress: 100%

Details:

- Show the username and id of each user
- Show the current points of each user
- Users should appear in score order, sub-sort by name when ties occur
- Pending-to-pick users should be marked accordingly
- Eliminated users should be marked accordingly

Part 1:

Progress: 100%

Details:

- Show various examples of points (3+ clients visible)
  - Include code snippets showing the code flow for this from server-side to UI
- Show that the sorting is maintained across clients
  - Include code snippets showing the code that handles this
- Show various examples of the pending-to-pick indicators
  - Include code snippets showing the code flow for this from server-side to UI
- Show various examples of elimination indicators
  - Include code snippets showing the code flow for this from server-side to UI

```
PointsPayload pointsPayload = new PointsPayload();
for (ServerThread player : activePlayers) {
    UserStatus status = new UserStatus(
        player.getClientName(),
        player.getClientId(),
        player.getUser().getPoints()
    );
    pointsPayload.addUserStatus(status);
}

Set<Long> eliminatedIds = new HashSet<>();
for (ServerThread p : roundLosers) {
    eliminatedIds.add(p.getClientId());
}
pointsPayload.setEliminatedIds(eliminatedIds);

Set<Long> pendingIds = new HashSet<>();
for (Map.Entry<ServerThread, String> entry : playerChoices.entrySet()) {
    if (entry.getValue() == null) {
        pendingIds.add(entry.getKey().getClientId());
    }
}
pointsPayload.setPendingPickIds(pendingIds);

for (ServerThread player : activePlayers) {
    player.sendToClient(pointsPayload);
}
```

onRoundStart

```
for (ServerThread loser : roundLosers) {
    eliminatedPlayers.add(loser);
    broadcastToActivePlayers(loser.getClientName() + " is eliminated!");
}
```

```
        List<ServerThread> remaining = new ArrayList<>();
        for (ServerThread player : activePlayers) {
            if (!eliminatedPlayers.contains(player)) {
                remaining.add(player);
            }
        }
```

Elimination

Ongame

GameOver

Saved: 8/3/2025 7:48:54 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for points updates from server-side to the UI
- Briefly explain the code flow for user list sorting
- Briefly explain the code flow for server-side to UI of pending-to-pick indicators
- Briefly explain the code flow for server-side to UI of elimination indicators

Your Response:

When the server sends updated points, it packages them in a PointsPayload which the client receives and passes to GameAreaUI.updateUserPanel(). This method updates the user list panel with the latest points and statuses. The user list is sorted inside UserListPanel.updateUserList() by points descending and then by name ascending. For pending-to-pick indicators, the server includes a list of user IDs in the payload which the client uses to mark those users as pending in the UI. Similarly, eliminated users are indicated by the server via a list of eliminated user IDs that the client uses to visually mark those users as eliminated in the UI.

the client uses to visually mark those users as eliminated in the UI.

Saved: 8/3/2025 7:48:54 PM

Task #2 (0.67 pts.) - Game Events Panel

Progress: 100%

Details:

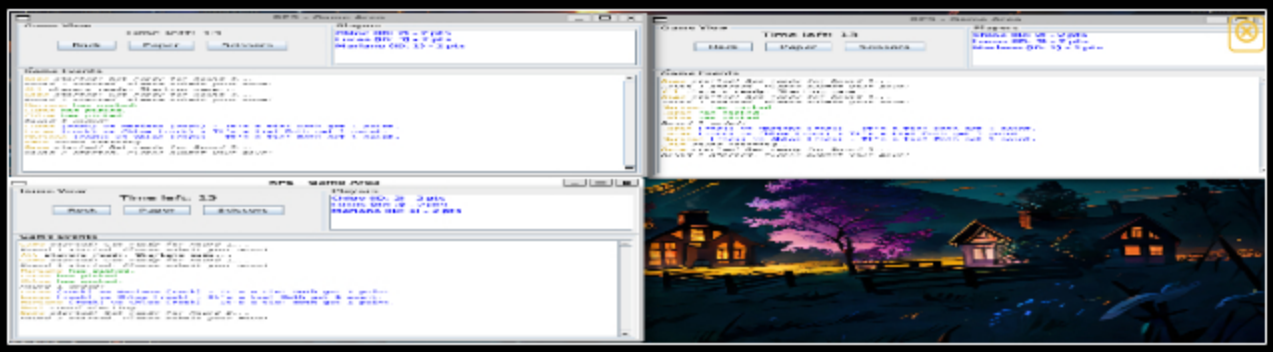
- Show the status of users picking choices
- Show the battle resolution messages from Milestone 2
  - Include messages about elimination
- Show the countdown timer for the round

Part 1:

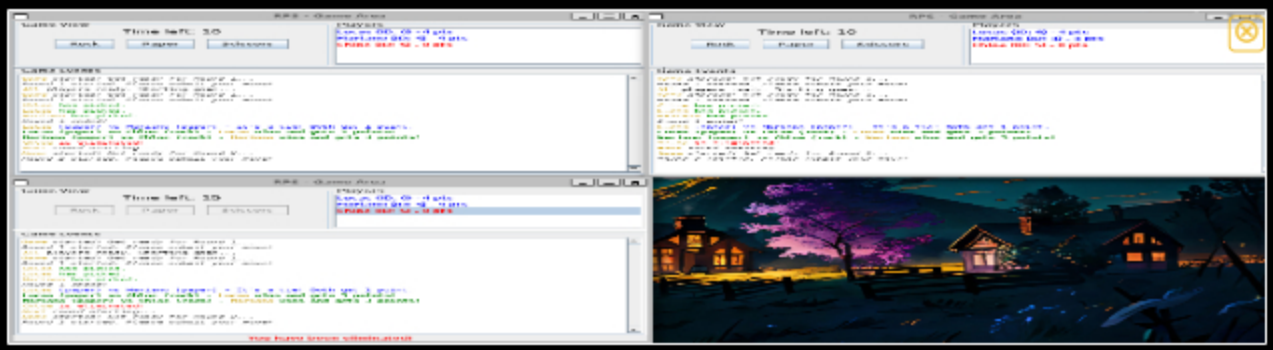
Progress: 100%

Details:

- Show various examples of each of the messages/visuals
- Show code snippets related to these messages from server-side to UI



GameGoing



Eliminations

```
public void handleReady(ServerThread sender, boolean isReady) {
    synchronized (this) {
        sender.setReady(isReady);

        broadcastReadyStatus(sender.getClientId(), isReady);
        broadcastToActivePlayers(sender.getClientName() + " is " + (isReady ? "READY" : "NOT RE

        long readyCount = activePlayers.stream().filter(ServerThread::isReady).count();
        int totalPlayers = activePlayers.size();
```

```
            gameStarted = true;
            broadcastToActivePlayers("All players ready. Starting game...");
            postGameCountdownStarted = false;
            onSessionStart();
        } else if (!postGameCountdownStarted && !gameStarted) {
            postGameCountdownStarted = true;
            startPostGameCountdown();
        }
    }
}
```

Code1

```
     * @return true for successful send
     */
    protected boolean sendMessage(long clientId, String message) {
        Payload payload = new Payload();
        payload.setPayloadType(PayloadType.MESSAGE);
        payload.setMessage(message);
        payload.setClientId(clientId);
        return sendToClient(payload);
    }
```

Code2

Saved: 8/5/2025 3:03:04 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for generating these messages and getting them onto the UI

Your Response:

The server triggers a message in GameRoom.java using broadcastToActivePlayers(), which calls sendMessage() on each ServerThread. This wraps the message in a Payload object with PayloadType.MESSAGE and sends it to the client via sendToClient(). On the client side, Client.java receives the payload through its socket listener. It detects the PayloadType.MESSAGE, extracts the message string, and calls GameAreaUI.logEvent(message) to display it in the game UI event panel.

Saved: 8/5/2025 3:03:04 AM

Task #3 (0.67 pts.) - Game Area

Progress: 100%

Details:

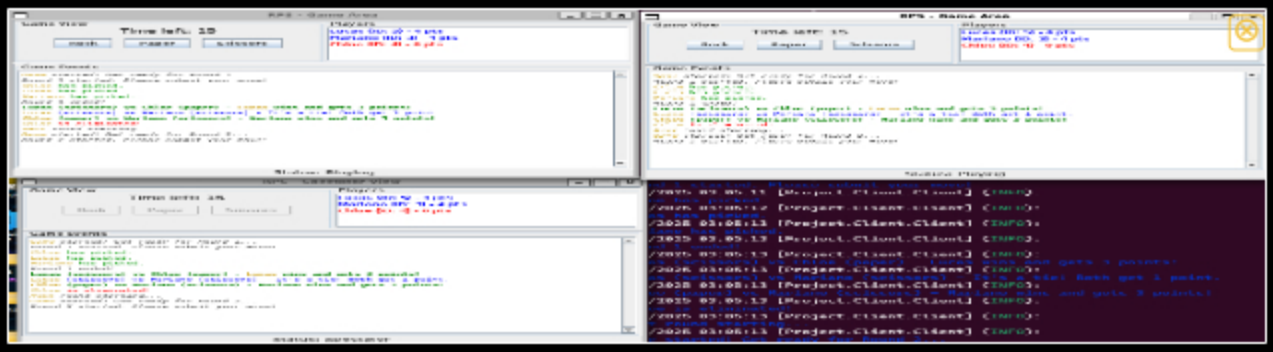
- UI should have components to allow the user to select their choice

Part 1:

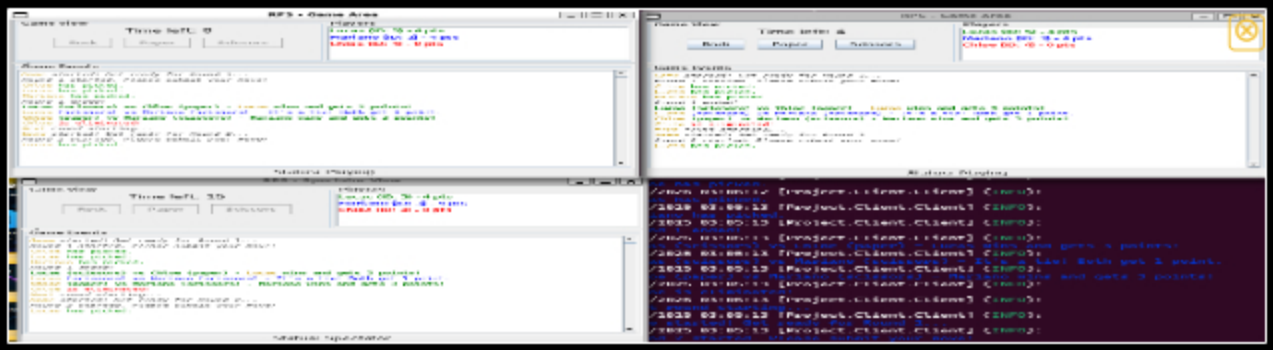
Progress: 100%

Details:

- Show various examples of selections across clients (3+ clients visible)
- Show the code related to sending choices upon selection
- Show the code related to showing visually what was selected



Example1



Example2

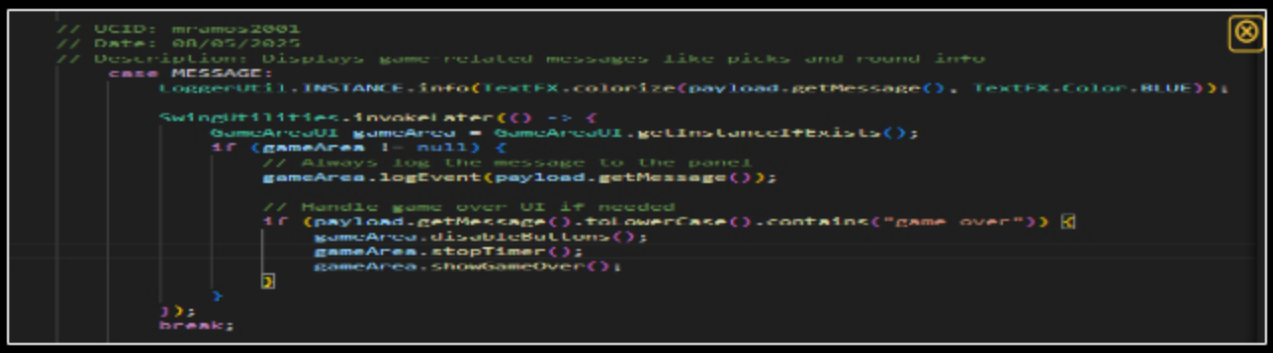
```
// UCID: mramos2001
// Date: 08/05/2025
// Description: Sends selected move to server on button press
rockButton = new JButton("Rock");
paperButton = new JButton("Paper");
scissorsButton = new JButton("Scissors");

rockButton.addActionListener(this::handlePick);
paperButton.addActionListener(this::handlePick);
scissorsButton.addActionListener(this::handlePick);

buttonPanel.add(rockButton);
buttonPanel.add(paperButton);
buttonPanel.add(scissorsButton);
gameViewPanel.add(buttonPanel, BorderLayout.CENTER);
```

Code1

```
// UCID: mramos2001
// Date: 08/05/2025
// Description: Sends CHOICE payload to server with player's move

public void sendChoice(String choice) {
    Payload payload = new Payload();
    payload.setPayloadType(PayloadType.CHOICE);
    payload.setChoice(choice);
    Client.INSTANCE.sendPayload(payload);

}
```

Code2

Code3



Code4

Saved: 8/5/2025 3:10:51 AM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for selecting a choice and having it reach the server-side
- Briefly explain the code flow for receiving the selection for the current player to update the UI

Your Response:

When a player clicks a move button in GameAreaUI, it calls sendChoice() which disables the buttons and sends the selected move to the server via Client.sendChoice(). The server receives the CHOICE payload in ServerThread, forwards it to GameRoom.handleChoice(), which validates and stores the move, then broadcasts a message like "Mariano has picked." On the client side, this message is received in Client.java, identified as a MESSAGE payload, and passed to GameAreaUI.logEvent(), which displays it in the game event panel as visual confirmation.

Saved: 8/5/2025 3:10:51 AM

Section #3: (4 pts.) Project Extra Features

Progress: 100%

Task #1 (2 pts.) - Extra Choices

Progress: 100%

Details:

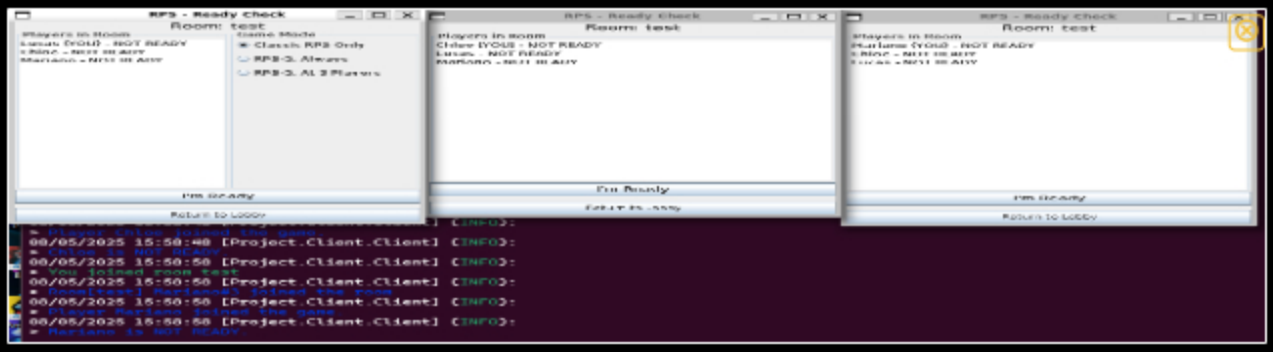
- Setting should be toggleable during Ready Check by session creator
  - (Option 1) Extra choices are available during the full session
  - (Option 2) Only activate extra options at different stages (i.e., last 3 players remaining)
- There should be at least 2 extra options for rps-5

Part 1:

Progress: 100%

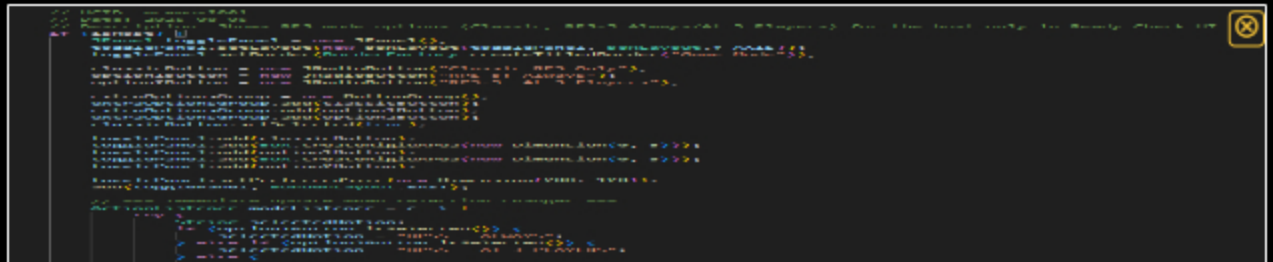
Details:

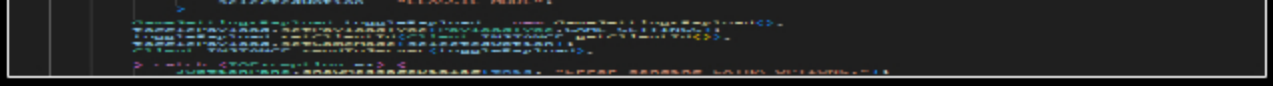
- Show the Ready Check screen with the option for the host (3+ clients must be visible)
  - Show the related code that makes this interactable only for the host
- Show the play screen with the extra options available
  - Show the related code for the UI and handling of these extra options (including battle logic)



ReadyHost

CurrentGame





Code1

```
// UCID: mramos2001
// Date: 2025-08-05
// Description: Enables Lizard and Spock buttons if game mode is RPS5 or one of the RPS5 conditional variants

private void updateMoveButtons() {
    String mode = currentGameMode.toUpperCase();

    boolean showExtras = mode.equals("RPS5") || mode.equals("RPS5 - ALWAYS") ||
        (mode.equals("RPS5 - AT 3 PLAYERS") && client.INSTANCE.getKnownClients().size() <= 3);

    if (lizardButton != null) {
        lizardButton.setVisible(showExtras);
        lizardButton.setEnabled(showExtras);
    }

    if (spockButton != null) {
        spockButton.setVisible(showExtras);
        spockButton.setEnabled(showExtras);
    }

    revalidate();
    repaint();
}
}
```

Code2

Saved: 8/5/2025 4:04:06 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling these options including how it's handled during the battle logic
- Note which option you went with in terms of activating the choices

Your Response:

The host sees radio buttons in ReadyCheckUI if Client.INSTANCE.isHost() is true. When the host selects a mode, a GAME_SETTINGS payload is immediately sent to the server with values like "CLASSIC", "RPS5", or "RPS5 - AT 3 PLAYERS". The client receives this in Client.java and calls GameAreaUI.setGameMode(mode). In GameAreaUI, updateMoveButtons() shows or hides Lizard and Spock based on the mode and player count. During battle, the server uses a beats(String a, String b) method that supports RPS-5 logic to determine winners. The implementation uses the "RPS5" option (RPS-5 always enabled) for activating Lizard and Spock.

Saved: 8/5/2025 4:04:06 PM

Task #2 (2 pts.) - Choice cooldown

Progress: 100%

Details:

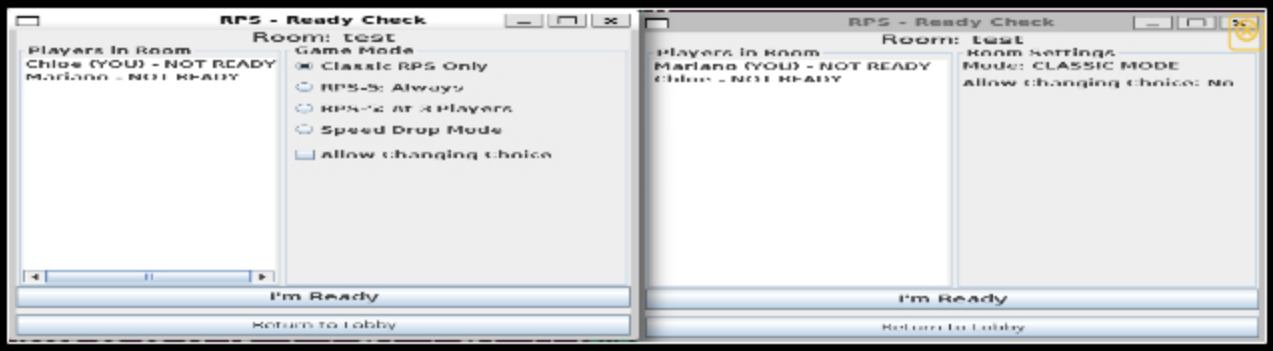
- Setting should be toggleable during Ready Check by session creator
- The choice on cooldown must be disable on the UI for the User

Part 1:

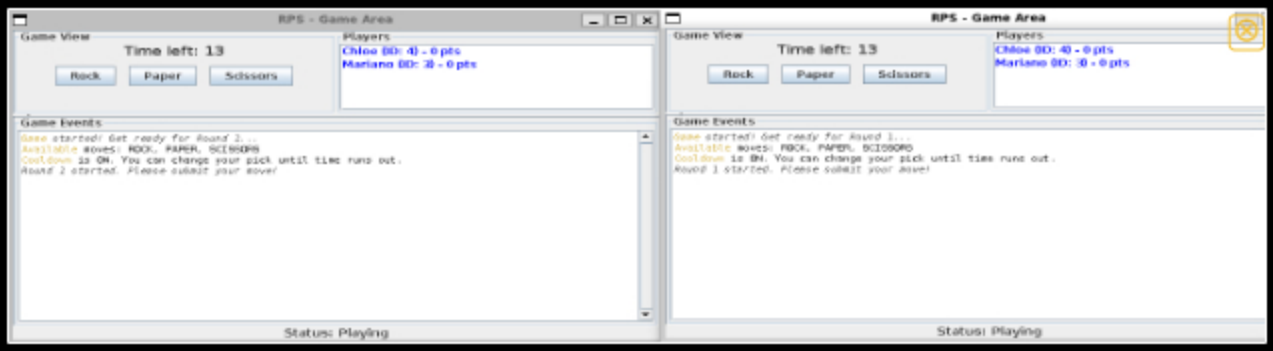
Progress: 100%

Details:

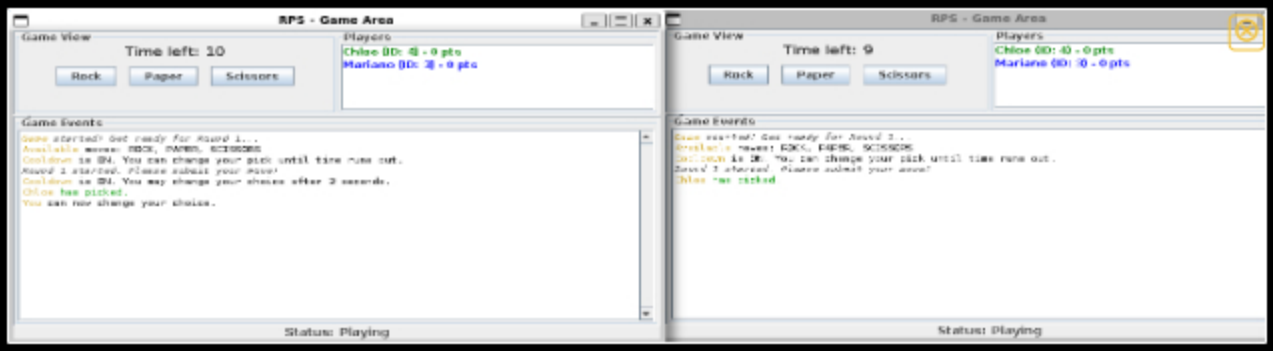
- Show the Ready Check screen with the option for the host (3+ clients must be visible)
  - Show the related code that makes this interactable only for the host
- Show a few examples of the play screen with the choice on cooldown
  - Show the related code for the UI and handling of the cooldown and server-side enforcing it



ReadyUI



InGame



InGame2

Code1

```
// UCID: mramos2001
// Date: 2025-08-05
// Description: Prevents client from picking again within 2 seconds if cooldown is enabled

public void setGameMode(String mode, boolean choiceCooldown, String setBy) {
    if (mode != null) {
        this.gameMode = mode;
        this.choiceCooldownEnabled = choiceCooldown;

        this.allowChoiceChanges = choiceCooldown;

        broadcastToActivePlayers("Game mode set to: " + mode + " by " + setBy);
        broadcastToActivePlayers("Choice cooldown is " + (choiceCooldown ? "ENABLED (you may change your pick)" : "DISABL

        GameSettingsPayload settings = new GameSettingsPayload();
        settings.setGameMode(mode);
        settings.setChoiceCooldown(choiceCooldown);
        for (ServerThread player : activePlayers) {
            player.sendToClient(settings);
        }
    }
}
```

Code2

Saved: 8/5/2025 10:17:37 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code for the host's option to toggle this feature
- Briefly explain the code related to handling and enforcing the cooldown period (include how this is recorded per user and reset when applicable)

Your Response:

The host toggles the cooldown via a checkbox in ReadyCheckUI, enabled only if Client.INSTANCE.isSessionHost() is true. This setting is sent to all players in a GameSettingsPayload. Cooldown is enforced client-side in GameAreaUI using a 2-second Timer that disables buttons after a pick. Server-side, GameRoom tracks each player's last pick time in lastPickTime. If a new pick arrives within 2 seconds, it's rejected and ignored. The timestamp resets only on valid picks.

Saved: 8/5/2025 10:17:37 PM

Section #4: (2 pts.) Project General Requirements

Progress: 100%

Task #1 (1 pt.) - Away Status

Progress: 100%

Details:

- Clients can mark themselves away and be skipped in turn flow but still part of the game
- The status should be visible to all participants
- A message should be relayed to the Game Events Panel (i.e., Bob is away or Bob is no longer away)
- The user list should have a visual representation (i.e., grayed out or similar)

Part 1:

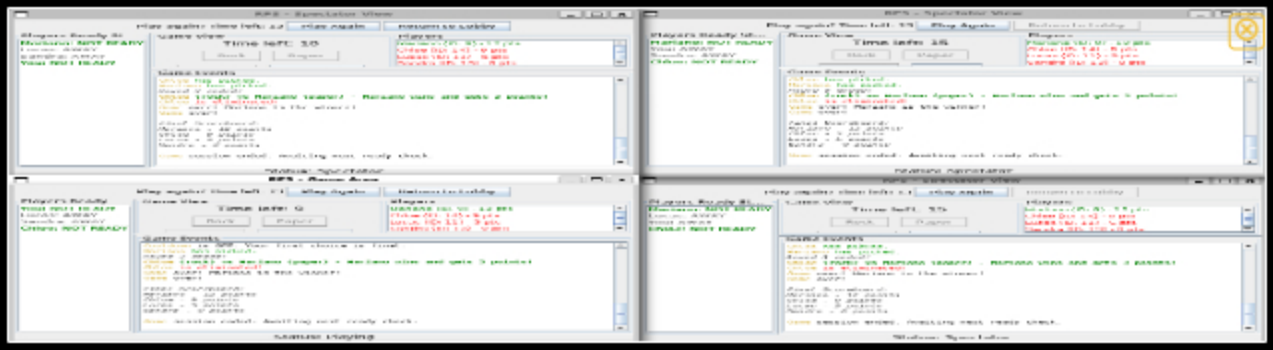
Progress: 100%

Details:

- Show the UI button to toggle away
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of away status (including Game Events Panel messages)
- Show the code that ignores an away user from turn/round logic

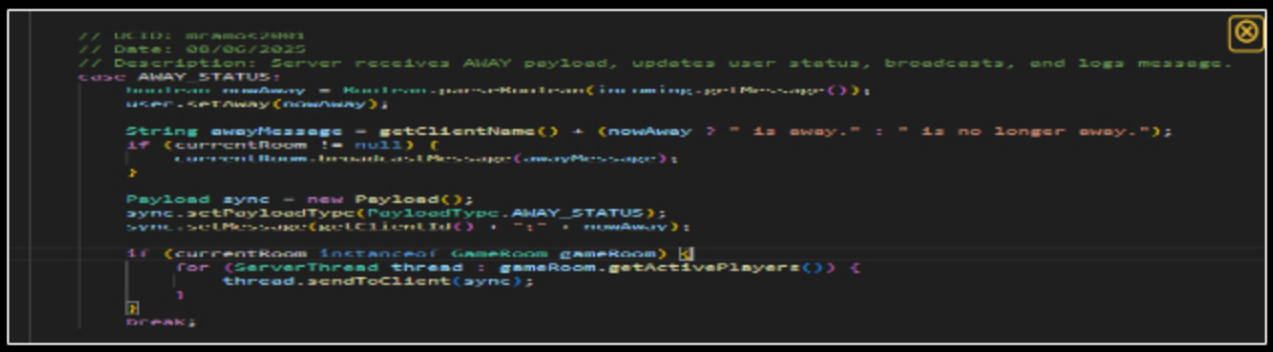
Sample1

Sample2



Sample3

Code1



```
// UCID: mra8c2hm1
// Date: 08/06/2025
// Description: Server receives AWAY payload, updates user status, broadcasts, and logs message.
case AWAY_STATUS:
    boolean nowAway = Boolean.parseBoolean(incoming.getMessage());
    user.setAway(nowAway);

    String awayMessage = getClientName() + (nowAway ? " is away." : " is no longer away.");
    if (currentRoom != null) {
        currentRoom.broadcastMessage(awayMessage);
    }

    Payload sync = new Payload();
    sync.setPayloadType(PayloadType.AWAY_STATUS);
    sync.setMessage(getClientId() + ":" + nowAway);

    if (currentRoom instanceof GameRoom gameRoom) {
        for (ServerThread thread : gameRoom.getActivePlayers()) {
            thread.sendToClient(sync);
        }
    }
    break;
```

Code2

Code3

Saved: 8/6/2025 8:52:31 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the away action from UI to server-side and back to UI
- Briefly explain how the server-side ignores the user from turn/round logic

Your Response:

When the player clicks the Away button in the UI, GameAreaUI toggles a local flag and sends an AWAY payload through Client.java to the server. The server receives this in ServerThread, updates the User's away status, sends updated points/statuses to all clients, and broadcasts a message payload so the Game Events Panel logs it. On the client side, Client.java updates the user list

(marking them as AWAY) and appends the status change to the Game Events Panel. On the server, in GameRoom.onRoundStart() and similar round methods, any player with user.isAway() is skipped when adding to playerChoices and never gets the START payload, ensuring they're excluded from the turn countdown and pick requirements while still remaining in the game.

Saved: 8/6/2025 8:52:31 PM

Task #2 (1 pt.) - Spectators

Progress: 100%

Details:

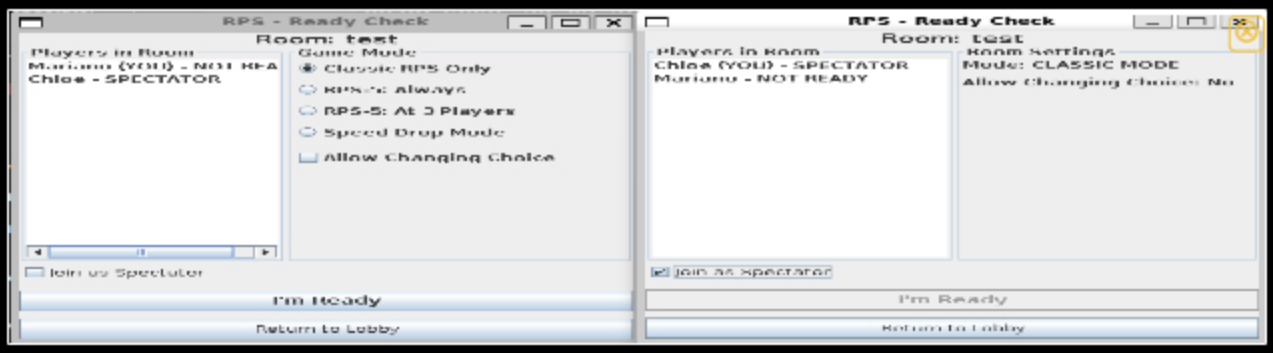
- Spectators are users who didn't mark themselves ready
  - Optionally you can include a toggle on the Ready Check page
- The can see all chat but are ignored from turn/round actions and can't send messages
- Spectators will have a visual representation in the user list to distinguish them from other players
- A message should be relayed to the Game Events Panel that a spectator joined (i.e., during an in-progress session)

Part 1:

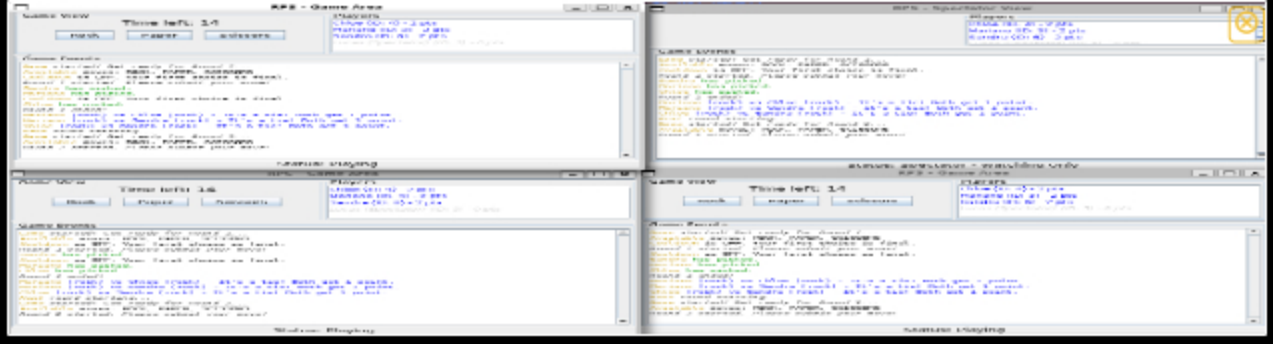
Progress: 100%

Details:

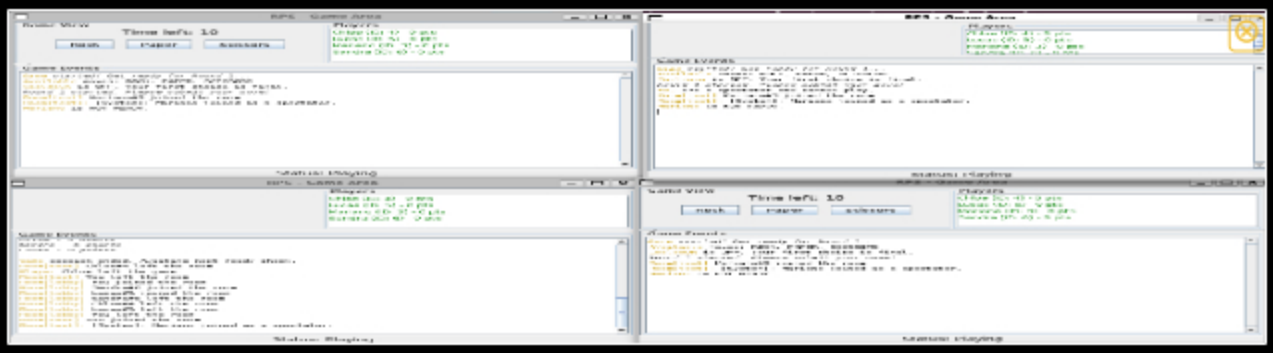
- Show the UI indicator of a spectator (visual and message)
- Show the related code flow from UI to server-side back to UI for showing the status
- Show the related code flow for sending the message to Game Events Panel
- Show various examples across 3+ clients of spectator status (including Game Events Panel messages)
- Show the code that ignores a spectator from turn/round logic
- Show the code that prevents spectators from sending messages (server-side)
- Show the spectator's view of the session
- Show the code related to the spectator seeing the session data (including things participants won't see)



Spec1

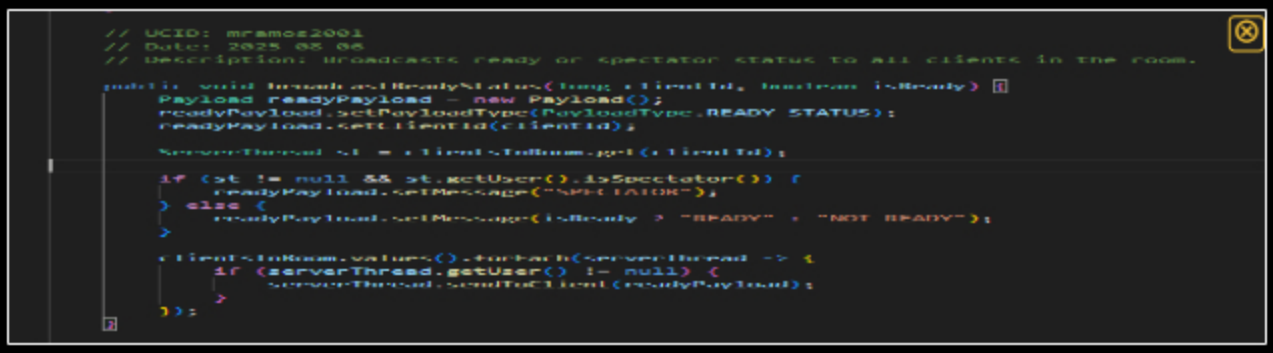


Spec2



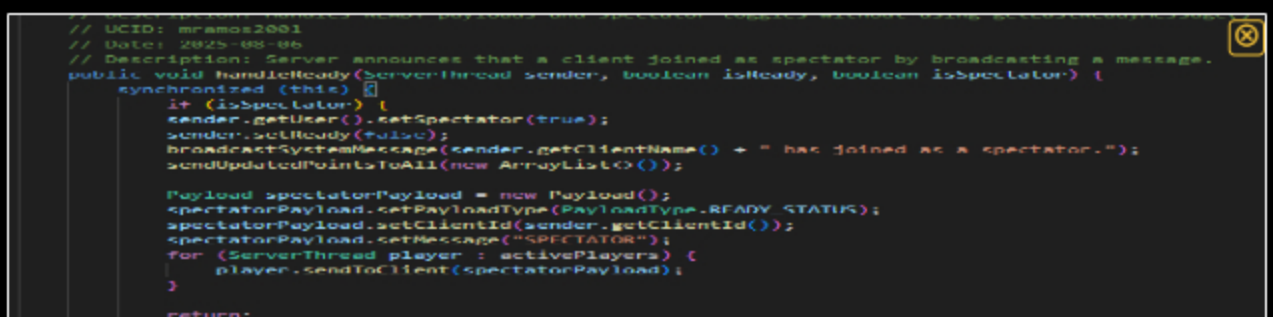
Spec3

Code1



```
// UCID: mramos2001
// Date: 2025 08 06
// Description: Broadcasts ready or spectator status to all clients in the room.

public void broadcastReadyStatus(long clientId, boolean isReady) {
    Payload readyPayload = new Payload();
    readyPayload.setPayloadType(PayloadType.READY_STATUS);
    readyPayload.setClientId(clientId);

    ServerThread st = clientsInRoom.get(clientId);

    if (st != null && st.getUser().isSpectator()) {
        readyPayload.setMessage("SPECTATOR");
    } else {
        readyPayload.setMessage(isReady ? "READY" : "NOT READY");
    }

    clientsInRoom.values().forEach(serverThread -> {
        if (serverThread.getUser() != null) {
            serverThread.sendToClient(readyPayload);
        }
    });
}
```

Code2



```
// UCID: mramos2001
// Date: 2025-08-06
// Description: Server announces that a client joined as spectator by broadcasting a message.
public void handleReady(ServerThread sender, boolean isReady, boolean isSpectator) {
    synchronized (this) {
        if (isSpectator) {
        sender.getUser().setSpectator(true);
        sender.setReady(false);
        broadcastSystemMessage(sender.getClientName() + " has joined as a spectator.");
        sendUpdatedPointsToAll(new ArrayList<>());

        Payload spectatorPayload = new Payload();
        spectatorPayload.setPayloadType(PayloadType.READY_STATUS);
        spectatorPayload.setClientId(sender.getClientId());
        spectatorPayload.setMessage("SPECTATOR");
        for (ServerThread player : activePlayers) {
            player.sendToClient(spectatorPayload);
        }

        return;
```

Code3

Saved: 8/6/2025 8:57:04 PM

Part 2:

Progress: 100%

Details:

- Briefly explain the code flow for the spectator logic from server-side and to UI
- Briefly explain how the server-side ignores the user from turn/round logic
- Briefly explain the logic that prevents spectators from sending a message
- Briefly explain the logic that shares extra details to the spectator (information normal participants won't see)

Your Response:

The server marks a user as a spectator upon mid-game join and broadcasts their spectator status to all clients, prompting the UI to disable player controls and show a spectator message. The client UI listens for this status and updates accordingly, showing the spectator view and logging relevant messages in the game events panel. During turn or round processing, the server simply skips any user flagged as a spectator, ensuring they do not participate in game actions. When a spectator tries to send messages, the server intercepts and ignores these inputs, preventing any spectator interaction with gameplay or chat. Additionally, the server shares detailed game state updates with spectators, including full player statuses and round information that normal participants may not see, allowing spectators a comprehensive view of the session without affecting gameplay.

Saved: 8/6/2025 8:57:04 PM

Section #5: (1 pt.) Misc

Progress: 100%

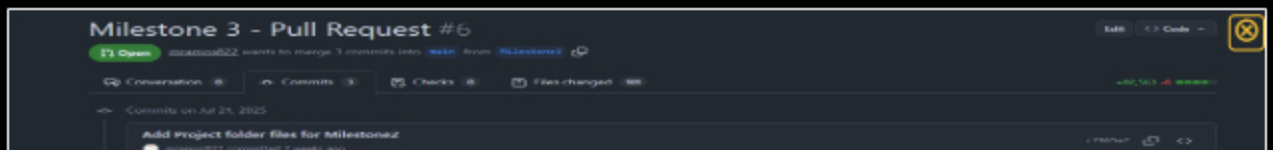
Task #1 (0.33 pts.) - Github Details

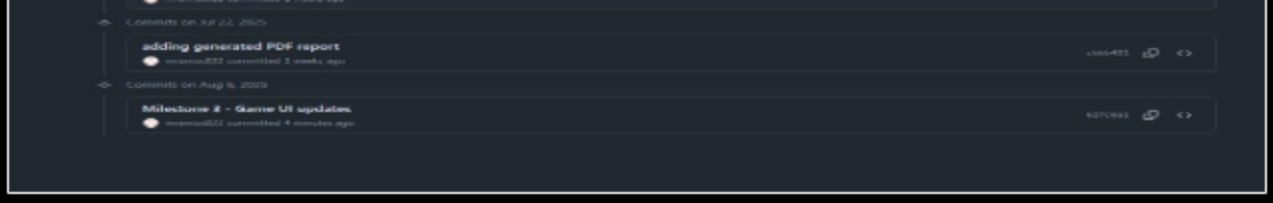
Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history





Comm

Saved: 8/6/2025 9:03:12 PM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request for Milestone3 to main (should end in `/pull/#`)

URL #1
https://github.com/mramos822/mr822-IT114-450/

URL
https://github.com/mramos822/n

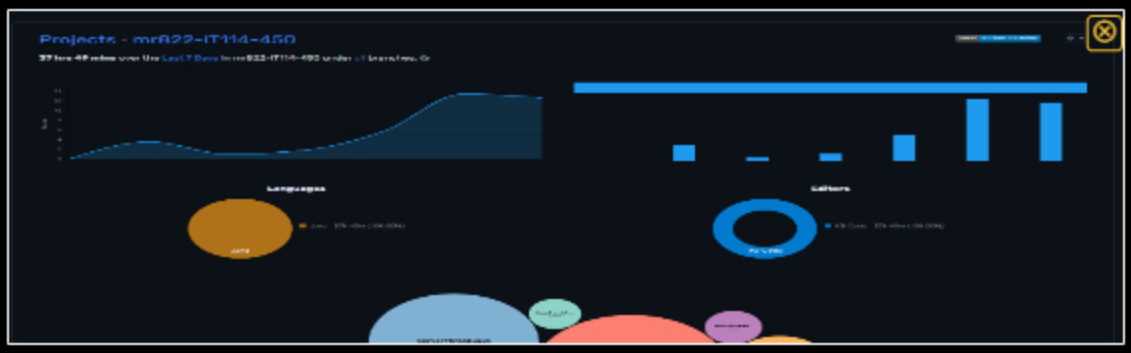
Saved: 8/6/2025 9:03:12 PM

Task #2 (0.33 pts.) - WakaTime - Activity

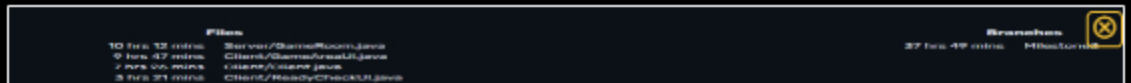
Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



SS1



SS2

Saved: 8/6/2025 9:01:54 PM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

During this milestone, I learned how to implement spectator functionality in a multiplayer game, including managing different user roles and syncing game states across clients. I also deepened my understanding of UI design for real-time multiplayer interactions, such as ready checks, player status indicators, and dynamic game event logging. Handling server-side logic to exclude spectators from gameplay while still providing them full visibility was a key learning point. Overall, this milestone helped me improve both client-server communication and user experience in a multiplayer environment.

Saved: 8/6/2025 9:02:09 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part was creating the connection and ready check UI panels. Building buttons and input fields was straightforward, and managing simple status

updates like “Ready” was easy once communication was working.

Saved: 8/6/2025 9:02:29 PM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was implementing the spectator logic correctly. It required careful handling to exclude spectators from game turns and messages while still allowing them full visibility of the game state. Ensuring smooth synchronization between server and multiple clients, especially with dynamic player statuses and UI updates, was challenging.

Saved: 8/6/2025 9:02:40 PM